

APOSTILA

Da User Story às Tasks

Como decompor itens do Product Backlog em trabalho executável pela equipe

Esta apostila apresenta um método prático para transformar User Stories (USs) de um Product Backlog em tasks técnicas e funcionais adequadas para uma Sprint. Como estudo de caso, será utilizado um Sistema de Dimensionamento Energético Residencial.

1. Product Backlog, User Story e Task: qual é a diferença?

O Product Backlog descreve o que o produto precisa entregar. A User Story representa uma necessidade do usuário ou do negócio. A task, por sua vez, descreve o trabalho concreto que a equipe precisa executar para implementar aquela necessidade.

Uma forma simples de visualizar a relação é:

Product Backlog → User Story → Tasks → Incremento do Produto

2. O que é uma User Story?

Uma User Story descreve uma funcionalidade a partir da perspectiva de quem obtém valor com ela. Uma estrutura bastante utilizada é:

Como [tipo de usuário], quero [objetivo], para [benefício/valor].

Exemplo:

Como usuário, quero cadastrar um imóvel para realizar seu dimensionamento energético.

A história informa o resultado esperado, mas ainda não explica todos os passos técnicos necessários para que esse resultado exista no software.

3. O que é uma Task?

Uma task é uma unidade de trabalho executável pela equipe. Ela deve indicar uma ação concreta, verificável e suficientemente pequena para ser desenvolvida e acompanhada durante a Sprint.

Exemplos de tasks associadas ao cadastro de imóvel:

- Criar estrutura/tabela de dados do imóvel.
- Implementar formulário de cadastro do imóvel.
- Implementar validação dos campos obrigatórios.
- Implementar rotina de gravação do imóvel no banco de dados.
- Implementar mensagem de confirmação do cadastro.
- Criar testes para o cadastro do imóvel.

4. User Story não deve ser confundida com Task

Aspecto	User Story	Task
Foco	Valor ou necessidade do usuário	Trabalho necessário para implementação
Linguagem	Funcional/negócio	Técnica e operacional
Escopo	Pode exigir várias atividades	Deve ser pequena e executável
Responsável	Equipe/produto	Um ou mais membros da equipe
Conclusão	Quando critérios de aceite são atendidos	Quando o trabalho específico está concluído

5. Método para transformar uma US em Tasks

Passo 1 — Compreender o resultado esperado

Antes de criar tasks, a equipe deve compreender o que a User Story realmente entrega ao usuário. Não se deve começar pela tecnologia. Primeiro deve estar claro qual comportamento será percebido no produto.

Passo 2 — Ler os critérios de aceite

Os critérios de aceite delimitam quando a história pode ser considerada correta. Eles ajudam a revelar validações, tratamentos de erro, persistência, cálculos e comportamentos que precisarão ser implementados.

Passo 3 — Identificar os componentes envolvidos

Pergunte quais partes do sistema serão afetadas: interface, regras de negócio, banco de dados, API, validações, cálculos, testes e integração entre componentes.

Passo 4 — Decompor o trabalho

Transforme cada necessidade técnica em uma ação objetiva. Prefira verbos como criar, implementar, validar, integrar, calcular, consultar, persistir e testar.

Passo 5 — Verificar se cada task é pequena e verificável

Uma task deve permitir que a equipe identifique claramente se foi concluída. Tasks excessivamente amplas, como 'fazer o cadastro', escondem trabalho e dificultam estimativas e acompanhamento.

Passo 6 — Verificar dependências

Determine a ordem lógica. Por exemplo, pode ser necessário criar a estrutura de dados antes de implementar a persistência; porém interface e banco podem ser desenvolvidos em paralelo quando o contrato entre eles estiver definido.

Passo 7 — Conferir cobertura

Ao terminar a decomposição, compare as tasks com a User Story e seus critérios de aceite. O conjunto das tasks deve ser suficiente para entregar a história completamente.

6. Exemplo completo: cadastro do imóvel

User Story

PB03 — Como usuário, quero cadastrar um imóvel para realizar seu dimensionamento energético.

Critério de aceite

Registrar identificação, endereço/localidade e tipo do imóvel.

Decomposição possível

- T01 — Definir os campos e tipos de dados da entidade Imóvel.
- T02 — Criar a tabela/estrutura de persistência de imóveis.
- T03 — Criar o formulário de cadastro do imóvel.
- T04 — Implementar validação dos campos obrigatórios.
- T05 — Implementar a rotina de gravação dos dados.
- T06 — Integrar o formulário à rotina de persistência.
- T07 — Exibir confirmação de cadastro realizado.
- T08 — Testar cadastro com dados válidos e inválidos.

7. Exemplo: registro de consumo mensal

PB05 — Como usuário, quero informar o consumo mensal de energia do imóvel.

Critério de aceite: aceitar consumo em kWh e rejeitar valores negativos ou inválidos.

- Definir a estrutura de dados do registro de consumo.
- Criar campo para entrada do consumo em kWh.
- Implementar validação numérica.
- Bloquear valores negativos.
- Associar o registro de consumo ao imóvel.
- Persistir o consumo no banco de dados.
- Testar valores válidos, negativos, vazios e não numéricos.

8. Exemplo: cálculo do maior consumo

PB08 — Como usuário, quero que o sistema identifique automaticamente o maior consumo mensal registrado.

Critério de aceite: calcular máximo = $\max(\text{consumos mensais})$ e apresentar o resultado em kWh/mês.

- Consultar os registros de consumo associados ao imóvel.
- Implementar função para identificar o maior valor de consumo.
- Tratar o caso de imóvel sem registros de consumo.
- Formatar o resultado em kWh/mês.
- Exibir o resultado na interface.
- Criar testes para diferentes conjuntos de valores.

9. Exemplo: consumo médio mensal

PB09 — Como usuário, quero visualizar também o consumo médio mensal para contextualizar o pico de consumo.

- Recuperar os consumos mensais cadastrados.

- Implementar a soma dos consumos.
- Calcular a média pela quantidade de meses registrados.
- Tratar ausência de registros para evitar divisão por zero.
- Apresentar o resultado em kWh/mês.
- Testar o cálculo com diferentes quantidades de registros.

10. Como reconhecer uma Task inadequada

Task inadequada	Problema	Forma mais adequada
Fazer o sistema.	Escopo amplo demais.	Criar formulário de cadastro do imóvel.
Fazer o banco.	Não indica qual estrutura deve ser criada.	Criar tabela de imóveis com os campos definidos na US.
Resolver PB08.	Apenas referencia a história.	Implementar função para obter o maior consumo mensal.
Testar tudo.	Não define o objeto do teste.	Testar rejeição de consumo mensal negativo.

11. Granularidade: até onde decompor?

Não existe um número obrigatório de tasks por User Story. A decomposição deve ser suficiente para tornar o trabalho visível e gerenciável, sem transformar atividades triviais em dezenas de itens administrativos.

Uma boa task tende a:

- ter um objetivo único e compreensível;
- produzir um resultado verificável;
- ser pequena em relação à duração da Sprint;
- permitir identificar dependências;
- contribuir diretamente para a conclusão da User Story.

12. Tasks técnicas também são necessárias

Nem todo trabalho aparece diretamente na interface. Uma User Story pode exigir tasks de banco de dados, regras de negócio, integração, testes ou segurança. Essas atividades são legítimas porque são necessárias para entregar o comportamento descrito pela história.

Entretanto, deve-se evitar transformar o Product Backlog em uma lista puramente técnica. A User Story continua representando valor; as tasks explicitam como a equipe realizará o trabalho.

13. Dependências entre Tasks

Considere a seguinte sequência para PB08:

Consultar consumos → Calcular máximo → Formatar resultado → Exibir resultado → Testar

Essa sequência evidencia dependências. A equipe não precisa necessariamente executar todas as tasks estritamente em série, mas deve conhecer quais resultados são pré-requisitos para os demais.

14. Definition of Done e conclusão da User Story

Concluir todas as tasks não significa automaticamente que a User Story está pronta. A história somente deve ser considerada concluída quando o incremento atende aos critérios de aceite e à Definition of Done adotada pela equipe.

Uma Definition of Done simples para o projeto poderia exigir:

- código implementado;
- funcionalidade integrada ao sistema;
- critérios de aceite atendidos;
- validações implementadas;
- testes executados sem erros conhecidos impeditivos;
- código versionado no repositório da equipe.

15. Modelo prático para refinamento

Para cada item do Product Backlog, a equipe pode utilizar a seguinte ficha:

User Story	Como _____, quero _____, para _____.
Critérios de aceite	1. _____ 2. _____ 3. _____
Componentes envolvidos	Interface / regra de negócio / dados / integração / testes / outros
Tasks	T01 _____ T02 _____ T03 _____ T04 _____
Dependências	_____
Condição de conclusão	Critérios de aceite + Definition of Done.

16. Atividade prática

Escolha três User Stories do Product Backlog do Sistema de Dimensionamento Energético Residencial. Para cada uma delas:

1. leia e interprete a User Story;
2. analise seus critérios de aceite;
3. identifique os componentes do sistema envolvidos;
4. crie entre quatro e oito tasks, quando essa quantidade fizer sentido;
5. identifique dependências entre as tasks;
6. verifique se o conjunto de tasks é suficiente para atender aos critérios de aceite;
7. registre as tasks no quadro da Sprint.

17. Checklist final

Antes de considerar a decomposição pronta, verifique:

- ☐ A equipe compreende o valor entregue pela User Story?
- ☐ Todas as tasks começam com uma ação clara?
- ☐ Cada task possui um resultado verificável?
- ☐ Há alguma task grande demais que deveria ser dividida?
- ☐ Existem tasks redundantes ou excessivamente pequenas?
- ☐ Banco de dados, regras, interface, validações e testes necessários foram considerados?
- ☐ As dependências estão claras?
- ☐ O conjunto de tasks atende a todos os critérios de aceite?
- ☐ Ao concluir as tasks, será possível entregar um incremento funcional?

Síntese

A decomposição de User Stories em tasks é uma etapa de planejamento do trabalho da equipe. A User Story responde principalmente ao que deve ser entregue e por que isso gera valor; as tasks respondem ao que precisa ser feito para tornar essa entrega realidade. Uma boa decomposição torna o trabalho visível, facilita estimativas, explicita dependências, melhora o acompanhamento da Sprint e reduz o risco de uma história aparentemente simples esconder atividades importantes.